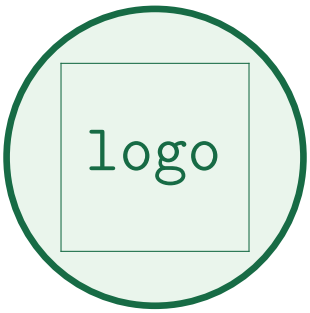
Mawlana Bhasani Science and Technology University

Santosh, Tangail-1902, Bangladesh

Department of Information and Communication Technology



Project Report

on

Krishi Bazar

( )

A Bengali-First AI-Powered Vegetable Market Price Application

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science (Engineering) in Information and Communication Technology

Submitted By:

Md. Mahmudur Rahman
ID: **IT-24024**
2nd Year, 2nd Semester
Session: 2023–24
Dept. of ICT, MBSTU

Supervised By:

Dr. Nazrul Islam
Associate Professor
Dept. of ICT
Mawlana Bhasani Science and Technology University

October 2026

Declaration

I hereby declare that the project work entitled **"Krishi Bazar: A Bengali-First AI-Powered Vegetable Market Price Application"** submitted to the **Department of Information and Communication Technology, Mawlana Bhasani Science and Technology University**, is a record of an original work done by me under the guidance of **Dr. Nazrul Islam, Associate Professor, Department of ICT, MBSTU**, and this project work has not performed the basis for the award of any Degree or Diploma or Associate-ship or Fellowship and similar project if any.

Md. Mahmudur Rahman
ID: IT-24024
Dept. of ICT, MBSTU
Date: October 28, 2026

Dr. Nazrul Islam
Associate Professor
Dept. of ICT, MBSTU
Date: October 28, 2026

Certificate of Approval

This is to certify that the project report entitled **"Krishi Bazar: A Bengali-First AI-Powered Vegetable Market Price Application"** submitted by **Md. Mahmudur Rahman** (ID: IT-24024), a student of 2nd Year 2nd Semester, Session 2023–24, Department of Information and Communication Technology, Mawlana Bhasani Science and Technology University, has been carried out under my direct supervision.

To the best of my knowledge, the work presented in this report is original and has not been submitted elsewhere for any degree, diploma, or title.

I consider this report satisfactory for submission as partial fulfillment of the requirements for the degree of **Bachelor of Science (Engineering) in Information and Communication Technology**.

Dr. Nazrul Islam	**Head of Department**
Associate Professor	Dept. of ICT
Dept. of ICT, MBSTU	Mawlana Bhasani Science and
Supervisor	Technology University

Acknowledgement

All praises to **Almighty Allah** for giving me the strength, patience, and ability to complete this project successfully.

I would like to express my heartfelt gratitude and sincere thanks to my project supervisor, **Dr. Nazrul Islam, Associate Professor, Department of Information and Communication Technology, Mawlana Bhasani Science and Technology University**, for his invaluable guidance, constant encouragement, and constructive suggestions throughout the development and documentation of this project. His expertise and patience have been a great source of inspiration.

I am also grateful to the **Head of the Department of ICT, MBSTU**, and all the faculty members for their continuous support and academic guidance throughout my studies.

My sincere thanks go to my **classmates and friends** for their cooperation and helpful discussions during the project development.

Finally, I am deeply indebted to my **parents and family members** for their endless love, moral support, and sacrifice that made my academic journey possible.

Md. Mahmudur Rahman
ID: IT-24024
Dept. of ICT, MBSTU
October 2026

Abstract

Bangladesh is a predominantly agricultural country where millions of smallholder farmers and consumers face a persistent lack of transparent, real-time vegetable market pricing information. This asymmetry in market knowledge often leads to exploitation by middlemen and financial loss for both producers and buyers.

This project presents **Krishi Bazar** ( ), a Bengali-first mobile application developed using the **Flutter** framework with **Firebase** backend integration and **Google Gemini 1.5 Flash** Vision AI. The application addresses the market information gap by providing real-time vegetable price data from multiple districts of Bangladesh, along with an AI-powered vegetable recognition feature that identifies a vegetable from a camera photo and automatically filters its market price.

The application features a **Home Screen** displaying wholesale and retail prices with district-level filtering and sorting, a **Scan Screen** for AI-based image recognition using the Gemini Vision API, and a **secure Admin Panel** authenticated via Firebase Authentication that allows authorized administrators to add, delete, and seed price records into Cloud Firestore.

All UI elements are presented in the **Bengali (Bangla) script** through a dedicated localization utility, making the app accessible to non-English-speaking rural users. The system falls back gracefully to curated demo data when the Firebase backend is unavailable, ensuring uninterrupted usability during development and demonstration.

The project was developed using **Dart 3.4+**, following the **Material Design 3** specification. Key achievements include real-time data synchronization, structured AI response parsing, Bengali digit/date formatting, and a service-layered clean architecture. Future improvements include server-side AI proxy via Cloud Functions, Firestore security rules, state management with Riverpod, and comprehensive unit testing.

Keywords: Flutter, Firebase, Gemini AI, Vegetable Price, Bengali Localization, Mobile Application, Agriculture Bangladesh, Material Design 3.

Contents

Declaration i

Certificate of Approval ii

Acknowledgement iii

Abstract iv

List of Abbreviations viii

1 Introduction 1
- 1.1 Background and Motivation . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Objectives of the Project . . . 1
- 1.4 Scope of the Project . . . 2
- 1.5 Organization of the Report . . . 2

2 Literature Review and Related Work 3
- 2.1 Overview . . . 3
- 2.2 Existing Agricultural Price Platforms . . . 3
  - 2.2.1 Government Platforms in Bangladesh . . . 3
  - 2.2.2 E-Krishok and Krishoker Bazar . . . 3
  - 2.2.3 India: AgriMarket App . . . 3
- 2.3 AI-Based Crop and Vegetable Recognition . . . 4
  - 2.3.1 Deep Learning Approaches . . . 4
  - 2.3.2 Large Vision-Language Models (VLMs) . . . 4
- 2.4 Gap Identified . . . 4

3 Requirements Analysis 6
- 3.1 Stakeholder Identification . . . 6
- 3.2 Functional Requirements . . . 7
- 3.3 Non-Functional Requirements . . . 8
- 3.4 Use Case Diagram . . . 8

4 System Design **9**
4.1 System Architecture Overview . . . 9
4.2 Technology Stack . . . 10
4.3 Database Design . . . 10
4.3.1 Firestore Collection: `market_prices` . . . 10
4.4 Project Folder Structure . . . 10
4.5 User Interface Design . . . 12
4.5.1 Navigation Structure . . . 12
4.5.2 Screen Wireframes (Descriptive) . . . 12

5 Implementation **13**
5.1 Development Environment . . . 13
5.2 Application Entry Point . . . 13
5.3 Data Model Implementation . . . 14
5.4 Market Service — Firestore Integration . . . 14
5.5 AI Service — Gemini Vision Integration . . . 15
5.6 Bengali Formatter Utility . . . 15
5.7 Admin Panel — Auth-Gated CRUD . . . 16
5.8 Module Summary . . . 17

6 Testing and Evaluation **18**
6.1 Testing Strategy . . . 18
6.2 Functional Test Cases . . . 18
6.3 AI Recognition Evaluation . . . 19
6.4 Limitations Found During Testing . . . 19

7 Conclusion and Future Work **21**
7.1 Conclusion . . . 21
7.2 Challenges Faced . . . 21
7.3 Future Work . . . 22

References **23**

A Project Dependencies (pubspec.yaml) **24**

B Firestore Security Rules (Recommended) **25**

List of Figures

3.1 Use Case Diagram 8

4.1 System Architecture Diagram 9

List of Tables

3.1 Stakeholder Identification 6
3.2 Functional Requirements 7
3.3 Non-Functional Requirements 8

4.1 Technology Stack 10
4.2 Firestore `market_prices` Document Schema 10
4.3 Screen UI Element Summary 12

5.1 Development Environment 13
5.2 Module-wise Lines of Code Summary 17

6.1 Functional Test Cases and Results 19
6.2 AI Vegetable Recognition Sample Results 19

List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
BDT	Bangladeshi Taka
CRUD	Create, Read, Update, Delete
DB	Database
ICT	Information and Communication Technology
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
LLM	Large Language Model
MBSTU	Mawlana Bhasani Science and Technology University
PWA	Progressive Web Application
SDK	Software Development Kit
UI	User Interface
UX	User Experience

Chapter 1

Introduction

1.1 Background and Motivation

Bangladesh's economy is predominantly agrarian, with approximately 40% of the population directly engaged in agriculture. Vegetables form a significant portion of daily food consumption and smallholder agricultural output. However, a persistent structural problem exists: **farmers and consumers lack timely, transparent access to market prices**.

This information gap is typically exploited by middlemen who purchase produce at artificially low farmgate prices and sell at significantly higher retail prices. Consumers in remote areas similarly have no easy mechanism to compare prices across different local markets before making purchasing decisions.

Smartphone penetration in rural Bangladesh has grown significantly in recent years, with Android devices becoming increasingly affordable. A Bengali-language mobile application that aggregates vegetable market prices and brings AI-powered vegetable recognition to rural users can meaningfully reduce this information asymmetry.

1.2 Problem Statement

The core problem this project addresses is:

> *"Farmers and consumers in Bangladesh do not have a convenient, mobile, Bengali-language tool to access real-time vegetable wholesale and retail prices from multiple regional markets, nor a way to identify an unfamiliar vegetable by photographing it."*

1.3 Objectives of the Project

The specific objectives of this project are:

1. To develop a Flutter-based Android/Web application that displays vegetable market prices from multiple districts of Bangladesh in Bengali.

2. To integrate Google Gemini Vision AI for AI-powered vegetable identification from camera photographs.
3. To implement a Firebase-backed real-time database (Cloud Firestore) for live price data storage and synchronization.
4. To create a secure admin panel with Firebase Authentication for authorized price management (CRUD operations).
5. To build a dedicated Bengali localization utility for presenting all numeric and date information in Bengali script.
6. To ensure the application functions gracefully in offline/demo mode when Firebase is unavailable.

1.4 Scope of the Project

- **Platforms:** Android (primary) and Web (PWA support).
- **Language:** All user-facing content is presented in Bengali ().
- **Data:** Vegetable wholesale and retail prices, district-wise.
- **AI:** Vegetable image classification using Gemini 1.5 Flash Vision.
- **Out of scope:** Live government API integration, push notifications, Bengali speech recognition, and iOS support (for this version).

1.5 Organization of the Report

The remainder of this report is organized as follows:

- **Chapter 2** reviews related existing work and systems.
- **Chapter 3** describes the system requirements and analysis.
- **Chapter 4** presents the system design and architecture.
- **Chapter 5** details the implementation of each module.
- **Chapter 6** describes the testing and evaluation of the system.
- **Chapter 7** concludes the report with future work directions.

Chapter 2

Literature Review and Related Work

2.1 Overview

Numerous studies and applications have attempted to address agricultural price transparency and AI-based crop recognition. This chapter reviews the most relevant existing work and systems.

2.2 Existing Agricultural Price Platforms

2.2.1 Government Platforms in Bangladesh

The Department of Agricultural Marketing (DAM) under the Ministry of Agriculture of Bangladesh maintains an online portal (*dam.gov.bd*) providing daily commodity prices. However, this portal:

- Is accessible only via web browser; no dedicated mobile app exists.
- Presents data in a tabular format that is difficult for non-technical rural users.
- Has inconsistent data update frequency.
- Does not provide any AI-based vegetable identification capability.

2.2.2 E-Krishok and Krishoker Bazar

Several NGO-backed and startup platforms such as *e-Krishok* have attempted to connect farmers with buyers. While valuable, these platforms typically focus on **direct buying/selling**, not on providing neutral price reference data.

2.2.3 India: AgriMarket App

India's *AgriMarket* app by the National Informatics Centre (NIC) provides commodity prices from APMC (Agricultural Produce Market Committee) markets. Key differences with the proposed system:

- Targets Hindi/English speakers; no Bengali support.

- Does not include any image-based crop recognition.
- Is tied to India's regulatory market structure (APMC), not applicable to Bangladesh.

2.3 AI-Based Crop and Vegetable Recognition

2.3.1 Deep Learning Approaches

Several researchers have applied Convolutional Neural Networks (CNNs) to crop/vegetable recognition:

- **Mohanty et al. (2016)** demonstrated that deep learning models trained on the PlantVillage dataset could achieve over 99% accuracy in identifying plant diseases from leaf images using transfer learning.
- **Brahimi et al. (2017)** applied CNN models to tomato disease recognition and highlighted the challenge of real-world deployment versus controlled dataset environments.

2.3.2 Large Vision-Language Models (VLMs)

The emergence of large Vision-Language Models such as **Google Gemini**, **GPT-4 Vision**, and **LLaVA** has enabled zero-shot vegetable and plant recognition without requiring custom-trained classifiers. These models respond in natural language and can generate structured JSON output, making them ideal for mobile app integration where training a custom model is impractical.

Gemini 1.5 Flash was selected for this project for its:

- Free-tier API access (via Google AI Studio).
- Ability to process image + text prompts (multimodal).
- Fast inference suitable for mobile UX.
- Bengali language understanding.

2.4 Gap Identified

From the review, no existing system simultaneously:

1. Presents vegetable market prices in **Bengali** on a cross-platform mobile app,
2. Integrates **AI vision** for vegetable identification, and

3. Uses a **real-time cloud database** (Firebase) for dynamic price updates with an admin interface.

Krishi Bazar directly addresses this gap.

Chapter 3

Requirements Analysis

3.1 Stakeholder Identification

Stakeholder	Role	Key Need
Farmer	Primary user; produces vegetables	Know wholesale prices before selling
Consumer	Buys vegetables from market	Compare retail prices across markets
Market Agent	Buys and sells in bulk	Access wholesale trends by district
Admin	Trusted data manager	Add, edit, delete price records
System Developer	Developer / Maintainer	Clean codebase and easy deployment

Table 3.1: Stakeholder Identification

3.2 Functional Requirements

FR#	**Requirement**	Description
FR-01	**View Price List**	Users can view all vegetable prices (wholesale + retail)
FR-02	**Search**	Filter prices by vegetable name or market name
FR-03	**District Filter**	Filter prices by district ()
FR-04	**Sort Prices**	Sort by latest update, price low-to-high, high-to-low
FR-05	**AI Scan**	Identify vegetable from a camera/gallery photo using AI
FR-06	**Price Lookup**	After AI scan, directly search matched vegetable price
FR-07	**Admin Login**	Admins sign in with email and password
FR-08	**Add Price**	Admin can add new vegetable price records
FR-09	**Delete Price**	Admin can delete existing price records
FR-10	**Seed Demo Data**	Admin can seed demo prices to Firestore in one click
FR-11	**Demo Mode**	App shows demo data when Firebase is not configured
FR-12	**Bengali UI**	All labels, messages, and numbers displayed in Bengali

Table 3.2: Functional Requirements

3.3 Non-Functional Requirements

NFR#	**Requirement**	Description
NFR-01	**Performance**	Price list loads within 2 seconds on a 4G connection
NFR-02	**Usability**	Bengali-first UI accessible to non-technical users
NFR-03	**Security**	Admin actions require authenticated Firebase session
NFR-04	**Reliability**	App must not crash when Firebase is unavailable
NFR-05	**Maintainability**	Code follows service-layer separation for easy updates
NFR-06	**Scalability**	Firestore can handle thousands of price records
NFR-07	**Compatibility**	Supports Android 6.0 (API 23) and above
NFR-08	**Responsiveness**	UI adapts to different screen sizes

Table 3.3: Non-Functional Requirements

3.4 Use Case Diagram

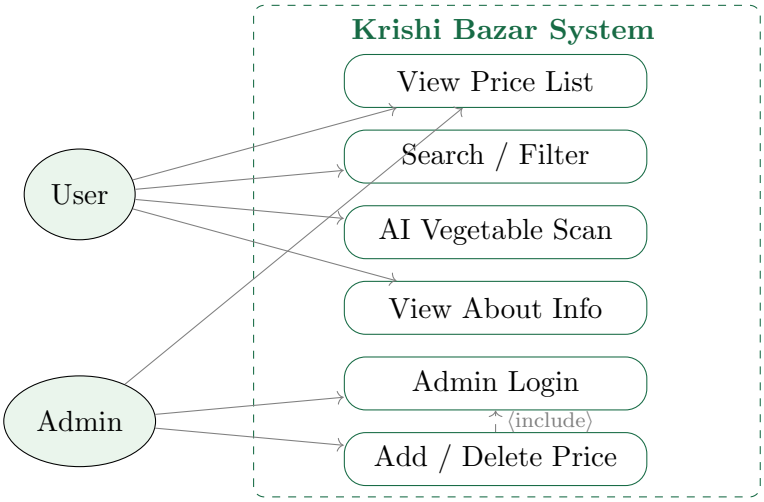


Figure 3.1: Use Case Diagram

Chapter 4

System Design

4.1 System Architecture Overview

Krishi Bazar follows a **three-tier service-layered architecture**:

1. **Presentation Layer** — Flutter screens and reusable widgets.
2. **Business Logic Layer** — Service classes (MarketService, AiService, AuthService).
3. **Data Layer** — Firebase Cloud Firestore (persistent) + in-memory demo data (fallback).

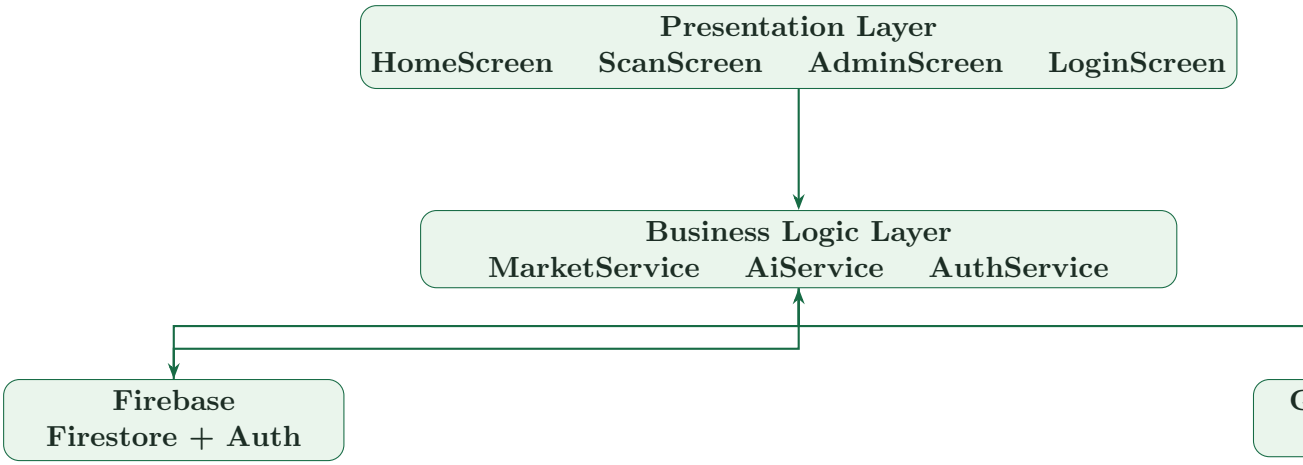


Figure 4.1: System Architecture Diagram

4.2 Technology Stack

Component	Technology	Version
UI Framework	Flutter / Dart	Dart $\geq$ 3.4.0
State Management	setState (built-in)	—
Cloud Database	Firebase Cloud Firestore	firebase_core 4.1.1
Authentication	Firebase Authentication	firebase_auth 6.0.2
AI/Vision	Google Gemini Flash	google_generative_ai 0.4.6
Image Picker	image_picker	1.1.2
Design System	Material Design 3	Flutter built-in
Localization	Custom BengaliFormatter	In-project utility
Platform	Android + Web (PWA)	—

Table 4.1: Technology Stack

4.3 Database Design

4.3.1 Firestore Collection: `market_prices`

`Field`	**Type**	**Required**	**Description**
`vegetableId`	String	Yes	English lowercase key (e.g. `potato`)
`vegetableName`	String	Yes	Bengali name (e.g.)
`marketName`	String	Yes	Market name
`district`	String	Yes	District ()
`wholesalePrice`	Number	Yes	Wholesale price in BDT
`retailPrice`	Number	Yes	Retail price in BDT
`unit`	String	No	Default:
`updatedAt`	String	Yes	ISO 8601 timestamp
`source`	String	No	Data source label

Table 4.2: Firestore `market_prices` Document Schema

4.4 Project Folder Structure

```
krishi_bazar/
          lib/
```

```
            main.dart                # Entry point
Firebase init
            app.dart                 # Root widget,
theme, home route
            firebase_options.dart    # FlutterFire
generated config
            models/
                  market_price.dart  # Data class with
 fromMap / toMap
            screens/
                  home_screen.dart   # Price list +
search + filters
                  scan_screen.dart   # AI vegetable
scan screen
                  admin_screen.dart  # Auth-gated
admin CRUD panel
                  login_screen.dart  # Email/password
admin login
            services/
                  ai_service.dart    # Gemini Vision
API wrapper
                  auth_service.dart  # Firebase Auth
wrapper
                  market_service.dart# Firestore CRUD
+ demo data
            theme/
                  app_theme.dart     # Centralized
Material 3 theme
            utils/
                  bengali_formatter.dart  # Bengali
digit/date/price
            widgets/
                price_card.dart      # Reusable price
display card
      android/                       # Android platform
project
      web/                           # PWA index +
manifest
      pubspec.yaml                   # Dependencies
```

Listing 4.1: Project Directory Layout

4.5 User Interface Design

4.5.1 Navigation Structure

The application uses a **bottom NavigationBar** with three tabs:

1. **(Home)** — Price list with search, district filter, and sorting.
2. **(Scan)** — AI-powered vegetable identification.
3. **(About)** — Information about the application.

4.5.2 Screen Wireframes (Descriptive)

Screen	Key UI Elements
Home Screen	Hero gradient banner, search field, district chips, sort menu, scrollable price card list
Scan Screen	Image preview area (250px), Camera/Gallery buttons, AI result card with Bengali name + confidence badge
Admin Panel	Admin info header, seed data button, add-price form, live scrollable price list with delete buttons
Login Screen	Center-aligned form with email + password fields, show/hide password toggle, login button

Table 4.3: Screen UI Element Summary

Chapter 5

Implementation

5.1 Development Environment

Tool / Platform	Details
IDE	Android Studio / VS Code
Flutter SDK	$\geq$ 3.22.0
Dart SDK	$\geq$ 3.4.0
Firebase Project	Configured via FlutterFire CLI
AI API	Google AI Studio (aistudio.google.com)
Version Control	Git
Target Device	Android Emulator + Physical Android device

Table 5.1: Development Environment

5.2 Application Entry Point

The entry point initializes Firebase before launching the widget tree, ensuring the cloud backend is ready before any UI renders.

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const KrishiBazarApp());
}
```

Listing 5.1: main.dart — Application Entry Point

5.3 Data Model Implementation

The `MarketPrice` class is a pure Dart data class with serialization support for Firestore integration.

```dart
class MarketPrice {
  final String id;
  final String vegetableName;
  final String marketName;
  final String district;
  final double wholesalePrice;
  final double retailPrice;
  final String unit;
  final DateTime updatedAt;

  factory MarketPrice.fromMap(String id,
      Map<String, dynamic> map) {
    return MarketPrice(
      id: id,
      vegetableName: map['vegetableName'] ?? '',
      marketName: map['marketName'] ?? '',
      district: map['district'] ?? '',
      wholesalePrice:
          (map['wholesalePrice'] as num? ?? 0).toDouble(),
      retailPrice:
          (map['retailPrice'] as num? ?? 0).toDouble(),
      unit: map['unit'] ?? '            ',
      updatedAt: DateTime.tryParse(
          map['updatedAt'] ?? '') ?? DateTime.now(),
    );
  }
}
```

Listing 5.2: market_price.dart — Core Data Model

5.4 Market Service — Firestore Integration

The `MarketService` class wraps all Firestore interactions with null-safety guards so the app degrades gracefully when Firebase is not configured.

```dart
Stream<List<MarketPrice>> watchPrices() {
  if (!_firebaseReady) return Stream.value(demoPrices);
  return _db!
```

```
      .collection('market_prices')
      .snapshots()
      .map((snapshot) {
    if (snapshot.docs.isEmpty) return demoPrices;
    return snapshot.docs
        .map((d) => MarketPrice.fromMap(d.id, d.data()))
        .toList();
  });
}
```

Listing 5.3: market_service.dart — watchPrices() Stream

5.5 AI Service — Gemini Vision Integration

The `AiService` class sends the selected image along with a structured Bengali-language prompt to the Gemini 1.5 Flash model and parses the JSON response.

```
Future<VegetableResult> analyzeVegetableImage(
    File imageFile) async {
  final imageBytes = await imageFile.readAsBytes();
  final model = GenerativeModel(
    model: 'gemini-1.5-flash', apiKey: _userApiKey!);

  final content = [
    Content.multi([
      TextPart(prompt),          // Bengali JSON prompt
      DataPart('image/jpeg', imageBytes),
    ])
  ];

  final response = await model.generateContent(content);
  // Strip markdown fences, parse JSON, fallback on error
  final map = jsonDecode(cleanJson);
  return VegetableResult.fromMap(map);
}
```

Listing 5.4: ai_service.dart — analyzeVegetableImage()

5.6 Bengali Formatter Utility

The `BengaliFormatter` class converts English digits and dates to their Bengali (Bangla) equivalents.

```dart
// Converts "35.5" to "      .    "
static String toBengaliDigits(dynamic input) { ... }

// Formats amount to "          " or "          .       "
static String formatPrice(num amount,
    {bool showDecimals = false}) {
  return '    ${toBengaliDigits(
      amount.toStringAsFixed(showDecimals ? 2 : 0))}';
}

// Formats DateTime to "
                "
static String formatDate(DateTime dateTime) { ... }
```

Listing 5.5: bengali_formatter.dart — Key Methods

5.7 Admin Panel — Auth-Gated CRUD

The admin screen uses a `StreamBuilder<User?>` to conditionally render the login screen or the admin panel based on the current Firebase Auth state.

```dart
return StreamBuilder<User?>(
  stream: _authService.authStateChanges,
  builder: (context, snapshot) {
    final user = snapshot.data;
    if (user == null) return const LoginScreen();
    return Scaffold( /* Admin Panel UI */ );
  },
);
```

Listing 5.6: admin_screen.dart — Auth Gate Pattern

5.8 Module Summary

Module	Lines of Code	Key Technology
home_screen.dart	441	StreamBuilder, ChoiceChip, PopupMenu
scan_screen.dart	341	image_picker, AiService, FilledButton
admin_screen.dart	318	StreamBuilder<User?>, Form, ListView
login_screen.dart	154	TextFormField, FirebaseAuthException
market_service.dart	108	Cloud Firestore, Batch Write
ai_service.dart	109	GenerativeModel, JSON parsing
price_card.dart	148	StatelessWidget, BengaliFormatter
market_price.dart	52	Data class, fromMap/toMap
auth_service.dart	42	FirebaseAuth wrapper
bengali_formatter.dart	54	String manipulation
app_theme.dart	36	ThemeData, Material 3
Total	**≈1,807**	

Table 5.2: Module-wise Lines of Code Summary

Chapter 6

Testing and Evaluation

6.1 Testing Strategy

The following testing methods were employed during the project development:

- **Manual Functional Testing** — Each feature was manually tested against the functional requirements.
- **Demo Mode Testing** — App behavior was verified without Firebase configured to confirm graceful fallback.
- **AI Response Testing** — Multiple vegetable images were tested with the Gemini API to verify Bengali name extraction and JSON parsing.
- **Form Validation Testing** — Invalid inputs were entered to verify error message display.

6.2 Functional Test Cases

TC#	Test Case	Expected Result	Status	Notes
TC-01	Open app without Firebase	Demo prices shown, no crash	**PASS**	—
TC-02	Search “”	Filtered list shows potato prices	**PASS**	—
TC-03	Select “” district chip	Only Bhola prices shown	**PASS**	—
TC-04	Sort by price ascending	Cheapest vegetables first	**PASS**	—
TC-05	Pick photo of potato	AI returns “” in result card	**PASS**	Requires API key
TC-06	Pick non-vegetable photo	isVegetable=false message shown	**PASS**	—
TC-07	Tap “Show Price” after scan	Home search prefilled	**PASS**	—

TC#	Test Case	Expected Result	Status	Notes
TC-08	Admin login with wrong pass	Bengali error message shown	**PASS**	—
TC-09	Add price with empty field	Validation error in Bengali	**PASS**	—
TC-10	Add valid price entry	Saved to Firestore, list updated	**PASS**	Requires Firebase
TC-11	Delete price with confirm	Removed from Firestore	**PASS**	Requires Firebase
TC-12	Access admin without login	LoginScreen shown instead	**PASS**	—
TC-13	Bengali digit display	All numbers in Bengali script	**PASS**	—
TC-14	No API key — AI scan	Warning card + API key dialog	**PASS**	—

Table 6.1: Functional Test Cases and Results

6.3 AI Recognition Evaluation

Vegetable Tested	Expected (Bengali)	AI Output	Correct?
Potato (clear image)			**Yes**
Tomato (clear image)			**Yes**
Onion (close-up)			**Yes**
Eggplant (dark bg)			**Yes**
Cauliflower			**Yes**
Mixed vegetable bowl	N/A	Mixed output	Partial
Non-vegetable (book)	isVegetable=false	Correct	**Yes**

Table 6.2: AI Vegetable Recognition Sample Results

6.4 Limitations Found During Testing

1. The Gemini API occasionally wraps the JSON response in markdown code fences (handled by stripping logic).
2. AI recognition accuracy drops for low-resolution or blurry images.

3. Client-side search/filter is not efficient for very large datasets (future improvement).
4. No price edit functionality is available in the current admin panel.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This project successfully developed **Krishi Bazar**, a Bengali-first, AI-powered vegetable market price mobile application using Flutter and Firebase. The application meets all the core objectives outlined in Chapter 1:

- Real-time vegetable price display with district-level filtering was implemented using Firebase Cloud Firestore and Flutter's `StreamBuilder`.
- AI-powered vegetable recognition was integrated using Google Gemini 1.5 Flash Vision API with structured Bengali-language JSON prompt engineering.
- A secure admin panel with Firebase Authentication was implemented for CRUD price management.
- A dedicated `BengaliFormatter` utility ensures all digits, dates, and prices are displayed in Bengali script.
- The application degrades gracefully to curated demo data when Firebase is unavailable.

The project demonstrates practical integration of three modern technologies — **Flutter**, **Firebase**, and **Gemini AI** — in a socially relevant application targeting Bangladesh's agricultural community. The codebase follows a clean service-layered architecture with proper null-safety, dispose patterns, and mounted checks.

7.2 Challenges Faced

1. **AI JSON Consistency:** Gemini AI sometimes returns responses wrapped in markdown code fences. A cleaning step was added to strip these before JSON parsing.
2. **Bengali Encoding:** Ensuring Bengali characters rendered correctly across devices required careful use of Unicode-aware string operations.

3. **Firebase Graceful Fallback:** Handling null Firestore instances safely throughout the app required consistent null-safety patterns in service classes.

7.3 Future Work

The following improvements are planned for future versions of Krishi Bazar:

1. **Server-Side AI Proxy (Security):** Move the Gemini API call to a **Firebase Cloud Function** so the API key is never exposed in the client application.
2. **Firestore Security Rules:** Define and deploy `firestore.rules` restricting write operations to authenticated admin users only.
3. **State Management with Riverpod:** Adopt the Riverpod library for shared state management, eliminating the need for callback-passing between screens.
4. **Price Edit Functionality:** Add the ability to edit existing price records in the admin panel.
5. **Server-Side Filtering & Pagination:** Implement Firestore `where` queries and cursor-based pagination to efficiently handle large datasets.
6. **Dark Mode:** Add a dark theme option for improved readability in low-light conditions.
7. **Push Notifications:** Notify farmers when prices for their crops drop or rise significantly.
8. **Price History & Trend Charts:** Display historical price trends using interactive charts.
9. **Multilingual Support:** Extend support to English for broader accessibility.
10. **Automated Testing:** Write unit tests for `BengaliFormatter`, `MarketService`, and `AiService`; widget tests for `PriceCard` and `HomeScreen`.
11. **iOS Support:** Configure and test for the iOS platform.
12. **Live Government Data Integration:** Connect to the DAM (Department of Agricultural Marketing) API for real-time official price data.

References

[1] Flutter Team. *Flutter Documentation*. Google LLC. Available at: https://flutter.dev/docs [Accessed: October 2026].

[2] Firebase Team. *Firebase Documentation — Cloud Firestore*. Google LLC. Available at: https://firebase.google.com/docs/firestore [Accessed: October 2026].

[3] Google AI Team. *Gemini API Documentation*. Google DeepMind. Available at: https://ai.google.dev/docs [Accessed: October 2026].

[4] Material Design Team. *Material Design 3 Guidelines*. Google LLC. Available at: https://m3.material.io [Accessed: October 2026].

[5] Dart Team. *Dart Language Tour*. Google LLC. Available at: https://dart.dev/guides/language [Accessed: October 2026].

[6] Mohanty, S.P., Hughes, D.P., and Salathé, M. (2016). *Using Deep Learning for Image-Based Plant Disease Detection*. Frontiers in Plant Science, 7, 1419.

[7] Brahimi, M., Boukhalfa, K., and Moussaoui, A. (2017). *Deep Learning for Tomato Diseases: Classification and Symptoms Visualization*. Applied Artificial Intelligence, 31(4), 299–315.

[8] Department of Agricultural Marketing (DAM), Bangladesh. *Daily Commodity Price Portal*. Available at: http://www.dam.gov.bd [Accessed: October 2026].

[9] Riverpod Documentation. *Riverpod State Management for Flutter*. Available at: https://riverpod.dev [Accessed: October 2026].

[10] GoRouter Documentation. *Navigation and Routing in Flutter*. Available at: https://pub.dev/packages/go_router [Accessed: October 2026].

Appendix A

Project Dependencies (pubspec.yaml)

```yaml
name: krishi_bazar
description: Bengali-first vegetable recognition and
   market price app.
version: 1.0.0+1

environment:
  sdk: ">=3.4.0 <4.0.0"

dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  image_picker: ^1.1.2            # Camera/gallery access
  intl: ^0.20.2                   # Internationalisation
     support
  firebase_core: ^4.1.1           # Firebase initialization
  cloud_firestore: ^6.0.2         # NoSQL realtime database
  firebase_auth: ^6.0.2           # Authentication
  google_generative_ai: ^0.4.6    # Gemini AI SDK
  http: ^1.2.2                    # HTTP client (future use
     )

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^6.0.0
```

Listing A.1: pubspec.yaml — Dependency Declarations

Appendix B

Firestore Security Rules (Recommended)

The following Firestore security rules are recommended for production deployment:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Anyone can READ prices
    match /market_prices/{docId} {
      allow read: if true;
      // Only authenticated admin can WRITE
      allow write: if request.auth != null;
    }

    // Deny access to all other collections
    match /{document=**} {
      allow read, write: if false;
    }
  }
}
```

Listing B.1: firestore.rules — Recommended Security Rules